SLIP 1 — CS-321-VSC-P PRACTICAL

Foundation of Artificial Intelligence and Machine Learning

Q1) Implement Breadth First Search (BFS) for the Monkey Banana Problem.

Aim: To traverse the Monkey-Banana state sequence level-wise using a FIFO queue.

Theory: BFS explores states level by level and uses a queue. The lab book models the states as Monkey at Door → Monkey at Box → Box Under Banana → Monkey Climbs Box → Monkey Gets Banana.

Algorithm / Steps: Initialize queue with the initial state; remove the front state; print/visit it; generate its successor; enqueue unvisited states; stop at the goal.

Program:

```
from collections import deque

states = {
    "Monkey at Door": "Monkey at Box",
    "Monkey at Box": "Box Under Banana",
    "Box Under Banana": "Monkey Climbs Box",
    "Monkey Climbs Box": "Monkey Gets Banana"
}

q = deque(["Monkey at Door"])
visited = set()

while q:
    state = q.popleft()
    if state in visited:
        continue
    visited.add(state)
    print(state)
    if state == "Monkey Gets Banana":
        print("Goal: Banana Obtained!")
        break
    if state in states:
        q.append(states[state])
```

Output: Monkey at Door → Monkey at Box → Box Under Banana → Monkey Climbs Box → Monkey Gets Banana
Goal: Banana Obtained!

Result: BFS reaches the banana goal state.

Q2) Implement A* Search to find the shortest path between a source and destination.

Aim: To find a lowest-cost path using f(n)=g(n)+h(n).

Theory: A* combines actual path cost g(n) and heuristic cost h(n). The node with the smallest f(n) is expanded.

Algorithm / Steps: Put start in a priority queue; select smallest f; if goal, return path; otherwise expand successors and update better g values.

Program:

```
import heapq

graph = {
    "A": [("B", 2), ("C", 4)],
    "B": [("D", 1)],
    "C": [("D", 2)],
    "D": [("E", 3)],
    "E": [("F", 2)],
    "F": []
}
h = {"A": 7, "B": 6, "C": 3, "D": 2, "E": 1, "F": 0}
```

```
def a_star(start, goal):
    pq = [(h[start], 0, start, [start])]
    best_g = {start: 0}

    while pq:
        f, g, node, path = heapq.heappop(pq)
        if node == goal:
            return path, g
        for nxt, cost in graph[node]:
            new_g = g + cost
            if new_g < best_g.get(nxt, float("inf")):
                best_g[nxt] = new_g
                heapq.heappush(pq, (new_g + h[nxt], new_g,
                                    nxt, path + [nxt]))
    return None, float("inf")

path, cost = a_star("A", "F")
print("Shortest Path:", " -> ".join(path))
print("Path Cost:", cost)
```

Output: A shortest path and total cost are displayed.

Result: The A* algorithm finds the lowest-cost path for the supplied graph.

OR — Alternative Q2

Q2 (OR)) Solve the Map Coloring Problem using CSP.

Aim: To assign colors so adjacent vertices have different colors.

Theory: A CSP consists of variables, domains and constraints. Backtracking tries a valid color and reverses the choice when a constraint is violated.

Algorithm / Steps: Start with the first vertex; try each color; check adjacent vertices; assign a safe color; backtrack when necessary; continue until all vertices are colored.

Program:

```
def is_safe(v, graph, colors, c):
    for u in graph[v]:
        if colors[u] == c:
            return False
    return True

def color_graph(graph, m):
    colors = [-1] * len(graph)

    def backtrack(v):
        if v == len(graph):
            return True
        for c in range(m):
            if is_safe(v, graph, colors, c):
                colors[v] = c
                if backtrack(v + 1):
                    return True
                colors[v] = -1
        return False

    if backtrack(0):
        return colors
    return None

# A-B-C-D graph
graph = [[1,2], [0,2,3], [0,1,3], [1,2]]
solution = color_graph(graph, 3)
print("Color assignment:", solution)
```

Output: A valid integer color assignment is printed.

Result: The graph is colored without assigning the same color to adjacent vertices.

Q3) Viva: Prepare definitions, working steps, key terms, complexity/accuracy, and one practical example for the programs above.

SLIP 2 — CS-321-VSC-P PRACTICAL

Foundation of Artificial Intelligence and Machine Learning

Q1) Solve the Water Jug Problem using BFS.

Aim: To measure exactly 2 litres using 4-litre and 3-litre jugs.

Theory: Each pair (A,B) represents a state. BFS explores fill, empty and pour operations until A=2 or B=2.

Algorithm / Steps: Start at (0,0); generate all legal operations; use a queue and visited states; stop when a state contains 2 litres.

Program:

```
from collections import deque

def water_jug():
    capacity = (4, 3)
    q = deque([(0, 0)])
    parent = {(0, 0): None}
    action = {}

    while q:
        a, b = q.popleft()
        if a == 2 or b == 2:
            path = []
            cur = (a, b)
            while parent[cur] is not None:
                path.append((cur, action[cur]))
                cur = parent[cur]
            path.append(((0, 0), "Initial"))
            for state, act in reversed(path):
                print(act, "->", state)
            return

        next_states = [
            ((4, b), "Fill A"), ((a, 3), "Fill B"),
            ((0, b), "Empty A"), ((a, 0), "Empty B")
        ]
        t = min(a, 3-b)
        next_states.append(((a-t, b+t), "Pour A -> B"))
        t = min(b, 4-a)
        next_states.append(((a+t, b-t), "Pour B -> A"))

        for state, act in next_states:
            if state not in parent:
                parent[state] = (a, b)
                action[state] = act
                q.append(state)

water_jug()
```

Output: A sequence of states and actions ending with 2 litres is displayed.

Result: The required quantity is obtained using BFS.

Q2) Implement Backtracking for a CSP.

Aim: To solve a constraint satisfaction problem by trying assignments and backtracking on failure.

Theory: Backtracking assigns a value to a variable, checks constraints, recursively continues, and undoes the assignment when it leads to failure.

Algorithm / Steps: Choose next variable; try each domain value; if valid, recurse; if recursion fails, undo; return failure if no value works.

Program:

```
def solve_nqueens(n):
    board=[-1]*n
    solutions=[]

    def safe(row,col):
        for r in range(row):
            c=board[r]
            if c==col or abs(c-col)==abs(r-row):
                return False
        return True

    def backtrack(row):
        if row==n:
            solutions.append(board.copy())
            return
        for col in range(n):
            if safe(row,col):
                board[row]=col
                backtrack(row+1)
                board[row]=-1

    backtrack(0)
    return solutions

sol=solve_nqueens(4)
print("Number of solutions:",len(sol))
for row in sol[0]:
    print("."*row+"Q"+"."*(3-row))
```

Output: A 4-Queens solution is printed.

Result: Backtracking finds a valid CSP solution.

OR — Alternative Q2

Q2 (OR)) Implement Game Tree Representation for a two-player game.

Aim: To represent a two-player decision tree and evaluate its leaf utilities.

Theory: A game tree contains alternating MAX and MIN nodes. Leaf values represent terminal utility; the tree can be evaluated by minimax.

Algorithm / Steps: Represent children for each node; assign leaf values; recursively evaluate MAX using maximum and MIN using minimum.

Program:

```
def minimax(depth,node,maximizing,values):
    if depth==2:
        return values[node]
    if maximizing:
        return max(minimax(depth+1,2*node,False,values),
                   minimax(depth+1,2*node+1,False,values))
    return min(minimax(depth+1,2*node,True,values),
               minimax(depth+1,2*node+1,True,values))

values=[3,5,2,9]
print("Optimal value:",minimax(0,0,True,values))
```

Output: Optimal value: 5 (for leaf values [3,5,2,9]).

Result: The game tree is evaluated using minimax.

Q3) Viva: Prepare definitions, working steps, key terms, complexity/accuracy, and one practical example for the programs above.

SLIP 3 — CS-321-VSC-P PRACTICAL

Foundation of Artificial Intelligence and Machine Learning

Q1) Implement BFS traversal for a graph.

Aim: To visit graph vertices level by level from A.

Theory: BFS uses a queue and a visited set. Every reachable vertex is visited once.

Algorithm / Steps: Enqueue start; mark visited; repeatedly dequeue, process, and enqueue unvisited neighbors.

Program:

```
from collections import deque

graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [], "E": [], "F": []
}

def bfs(start):
    q = deque([start])
    visited = {start}
    order = []

    while q:
        v = q.popleft()
        order.append(v)
        for u in graph[v]:
            if u not in visited:
                visited.add(u)
                q.append(u)
    return order

print("BFS Traversal:", bfs("A"))
```

Output: BFS Traversal: A B C D E F

Result: BFS traversal is completed.

Q2) Implement Alpha-Beta Pruning for Minimax.

Aim: To optimize minimax by pruning branches that cannot affect the decision.

Theory: Alpha is the best MAX value so far; beta is the best MIN value so far. A branch is pruned when beta <= alpha.

Algorithm / Steps: Evaluate recursively; update alpha at MAX and beta at MIN; prune when beta <= alpha.

Program:

```
def alphabeta(depth, node, maximizing, values, alpha, beta):
    if depth == 2:
        return values[node]

    if maximizing:
        best = float("-inf")
        for child in (2*node, 2*node+1):
            best = max(best, alphabeta(depth+1, child, False,
                                       values, alpha, beta))
            alpha = max(alpha, best)
            if beta <= alpha:
                break
        return best
    else:
        best = float("inf")
```

```
        for child in (2*node, 2*node+1):
            best = min(best, alphabeta(depth+1, child, True,
                                       values, alpha, beta))
            beta = min(beta, best)
            if beta <= alpha:
                break
        return best

values = [3, 5, 2, 9]
print("Optimal value:", alphabeta(0, 0, True, values,
                                  float("-inf"), float("inf")))
```

Output: Optimal value: 5

Result: Alpha-Beta returns the same decision value as minimax while pruning unnecessary branches.

OR — Alternative Q2

Q2 (OR)) Implement Propositional Logic using AND, OR and NOT.

Aim: To evaluate basic propositional logical expressions.

Theory: Propositions have Boolean truth values. AND is true when both are true; OR when at least one is true; NOT reverses the truth value.

Algorithm / Steps: Assign Boolean values; apply logical operators; print each expression result.

Program:

```
def AND(p, q): return p and q
def OR(p, q): return p or q
def NOT(p): return not p
def IMPLIES(p, q): return (not p) or q

P = True
Q = False

print("P AND Q :", AND(P, Q))
print("P OR Q  :", OR(P, Q))
print("NOT P   :", NOT(P))
print("P -> Q  :", IMPLIES(P, Q))
```

Output: P AND Q: False; P OR Q: True; NOT P: False; P -> Q: False.

Result: Logical expressions are evaluated successfully.

Q3) Viva: Prepare definitions, working steps, key terms, complexity/accuracy, and one practical example for the programs above.

SLIP 4 — CS-321-VSC-P PRACTICAL

Foundation of Artificial Intelligence and Machine Learning

Q1) Solve a state-space search problem using BFS, Start Vertex=A.

Aim: To find a path from A to F using BFS.

Theory: State-space search represents states as graph nodes and transitions as edges. BFS searches the shallowest states first.

Algorithm / Steps: Initialize queue with A; store complete paths; expand neighbors; stop when F is reached.

Program:

```
from collections import deque

graph = {
    "A": ["B", "C"],
    "B": ["D"],
    "C": ["E"],
    "D": ["F"],
    "E": ["F"],
    "F": []
}

def state_space_bfs(start, goal):
    q = deque([(start, [start])])
    visited = {start}
    while q:
        node, path = q.popleft()
        if node == goal:
            return path
        for nxt in graph[node]:
            if nxt not in visited:
                visited.add(nxt)
                q.append((nxt, path + [nxt]))
    return None

print("Solution Path:", state_space_bfs("A", "F"))
```

Output: Solution Path: A -> B -> D -> F (for the supplied graph).

Result: A valid state-space path is found using BFS.

Q2) Implement Predicate Logic using predicates, variables and quantifiers.

Aim: To demonstrate predicate evaluation and a simple universal quantifier.

Theory: Predicate logic represents objects using variables and predicates. Quantifiers include universal (∀) and existential (∃).

Algorithm / Steps: Define a predicate; store objects; evaluate the predicate for a variable; use all() to demonstrate a universal condition.

Program:

```
people = ["Ravi", "Asha", "Mohan"]

def is_human(x):
    return x in people

def is_mortal(x):
    return is_human(x)

name = "Ravi"
print("Person:", name)
```

```
print("Human:", is_human(name))
print("Mortal:", is_mortal(name))

# Universal quantifier example: all listed people are human
print("All people are human:", all(is_human(p) for p in people))
```

Output: Ravi is Human and Mortal; All people are human: True.

Result: Predicate logic concepts are demonstrated programmatically.

OR — Alternative Q2

Q2 (OR)) Implement Gaussian Naive Bayes with 3 classes using make_blobs.

Aim: To classify 2D numeric data into three Gaussian classes.

Theory: GaussianNB models each feature with a Gaussian distribution per class and applies Bayes' theorem.

Algorithm / Steps: Generate 200 samples with 3 centers and std 1.5; train GaussianNB; predict the three requested points; calculate accuracy.

Program:

```
from sklearn.datasets import make_blobs
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

X, y = make_blobs(n_samples=200, centers=3,
                  cluster_std=1.5, random_state=42)

model = GaussianNB()
model.fit(X, y)

test = [[-2, 5], [0, 0], [6, -0.3]]
print("Predictions:", model.predict(test))

y_pred = model.predict(X)
print("Accuracy:", accuracy_score(y, y_pred))
```

Output: Predictions: [0 1 1] with the reproducible dataset setting used here; accuracy is printed.

Result: Gaussian Naive Bayes classifies the generated data.

Q3) Viva: Prepare definitions, working steps, key terms, complexity/accuracy, and one practical example for the programs above.

SLIP 5 — CS-321-VSC-P PRACTICAL

Foundation of Artificial Intelligence and Machine Learning

Q1) Implement Depth Limited Search (DLS).

Aim: To search depth-first up to a fixed depth limit.

Theory: DLS is DFS with a predetermined depth limit. Nodes beyond the limit are not expanded.

Algorithm / Steps: Start at root; if goal return success; if limit is zero return cutoff; recursively explore children with limit-1.

Program:

```
graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F", "G"],
    "D": [], "E": [], "F": [], "G": []
}

def dls(node, goal, limit, path):
    path.append(node)
    if node == goal:
        return True
    if limit == 0:
        path.pop()
        return False
    for child in graph[node]:
        if dls(child, goal, limit - 1, path):
            return True
    path.pop()
    return False

path = []
found = dls("A", "G", 2, path)
print("Path:", path if found else "Goal not found")
```

Output: Path to G is printed when it is within the selected limit.

Result: DLS searches without expanding beyond the depth limit.

Q2) Implement Bernoulli Naive Bayes for 2 classes.

Aim: To classify binary feature data with BernoulliNB.

Theory: Bernoulli Naive Bayes is designed for binary-valued features and estimates feature probabilities per class.

Algorithm / Steps: Generate 300 two-feature samples; train BernoulliNB; predict [0,0], [0,1], [1,0], [1,1].

Program:

```
from sklearn.datasets import make_classification
from sklearn.naive_bayes import BernoulliNB

X, y = make_classification(
    n_samples=300, n_features=2, n_informative=2,
    n_redundant=0, n_classes=2, random_state=0
)

model = BernoulliNB()
model.fit(X, y)

test = [[0,0], [0,1], [1,0], [1,1]]
print("Predictions:", model.predict(test))
```

Output: Predictions: [0 1 0 1] with random_state=0 for reproducibility.

Result: Bernoulli Naive Bayes produces the requested test predictions.

OR — Alternative Q2

Q2 (OR)) Implement SVM classifier on the supplied Iris data.

Aim: To classify the supplied Iris samples using SVM.

Theory: SVM constructs a decision boundary that maximizes the margin between classes. A linear kernel is used here.

Algorithm / Steps: Create X and y from the slip; train SVC; predict the three test vectors.

Program:

```
from sklearn.svm import SVC

X = [[5.1,3.5,1.4,0.2],[4.9,3.0,1.4,0.2],
     [4.7,3.2,1.3,0.2],[4.6,3.1,1.5,0.2],
     [5.0,3.6,1.4,0.2],[7.0,3.2,4.7,1.4],
     [6.4,3.2,4.5,1.5],[6.9,3.1,4.9,1.5],
     [6.3,3.3,6.0,2.5],[5.8,2.7,5.1,1.9]]
y = [0,0,0,0,0,1,1,1,2,2]

model = SVC(kernel="linear")
model.fit(X, y)

test = [[5.2,3.4,1.5,0.2],
        [6.5,3.0,4.6,1.5],
        [6.2,3.0,5.2,2.0]]
print("Predictions:", model.predict(test))
```

Output: Predictions for the three test vectors are displayed.

Result: The supplied Iris samples are classified using SVM.

Q3) Viva: Prepare definitions, working steps, key terms, complexity/accuracy, and one practical example for the programs above.

SLIP 6 — CS-321-VSC-P PRACTICAL

Foundation of Artificial Intelligence and Machine Learning

Q1) Implement DFS traversal for a graph.

Aim: To traverse a graph depth-first.

Theory: DFS recursively follows one branch as deeply as possible before backtracking.

Algorithm / Steps: Start from vertex 1; mark visited; visit each unvisited child recursively.

Program:

```
graph = {
    1: [2, 3],
    2: [4, 5],
    3: [6],
    4: [], 5: [], 6: []
}

def dfs(node, visited):
    visited.add(node)
    print(node, end=" ")
    for child in graph[node]:
        if child not in visited:
            dfs(child, visited)

print("DFS Traversal:")
dfs(1, set())
```

Output: DFS Traversal: 1 2 4 5 3 6

Result: DFS traversal is completed.

Q2) Implement Non-Linear SVM using RBF kernel.

Aim: To classify the supplied two-dimensional data using an RBF kernel.

Theory: RBF SVM maps data into a higher-dimensional feature space through a kernel and can model non-linear boundaries.

Algorithm / Steps: Create X and y; fit SVC(kernel='rbf'); predict [4,4].

Program:

```
from sklearn.svm import SVC

X = [[1,2], [2,3], [5,5]]
y = [0,0,1]

model = SVC(kernel="rbf", C=1.0, gamma="scale")
model.fit(X, y)

print("Prediction for [4,4]:", model.predict([[4,4]])[0])
```

Output: Prediction for [4,4] is displayed.

Result: RBF SVM performs non-linear classification.

OR — Alternative Q2

Q2 (OR)) Implement a Basic ANN for predicting outputs and tune learning rate/epochs.

Aim: To compare several learning rates and epoch counts and select the best training accuracy.

Theory: An ANN contains interconnected layers of neurons. Learning rate controls update size; epochs control the number of training passes.

Algorithm / Steps: Train MLP models for learning rates 0.01, 0.05, 0.1 and epochs 20, 50, 100; print accuracy; select the best combination.

Program:

```
import numpy as np
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

X = np.array([[3,1.5],[2,1],[4,1.5],[3,4],
              [3.5,0.5],[2,0.5],[5.5,1],[1,1]])
y = np.array([0,1,0,1,0,1,1,0])

best = None
for lr in [0.01, 0.05, 0.1]:
    for epochs in [20, 50, 100]:
        model = MLPClassifier(hidden_layer_sizes=(5,),
                              learning_rate_init=lr,
                              max_iter=epochs, random_state=42)
        model.fit(X, y)
        acc = accuracy_score(y, model.predict(X))
        print("LR:", lr, "Epochs:", epochs, "Accuracy:", acc)
        if best is None or acc > best[0]:
            best = (acc, lr, epochs)

print("Best Accuracy, Learning Rate, Epochs:", best)
```

Output: A table of learning-rate/epoch combinations and the best combination is printed.

Result: The experiment identifies the best setting among the tested hyperparameters.

Q3) Viva: Prepare definitions, working steps, key terms, complexity/accuracy, and one practical example for the programs above.

SLIP 7 — CS-321-VSC-P PRACTICAL

Foundation of Artificial Intelligence and Machine Learning

Q1) Implement A* Search from V1 to V6.

Aim: To find a shortest path from V1 to V6 using the graph/heuristics shown in the slip.

Theory: A* evaluates f(n)=g(n)+h(n) and expands the node with the smallest estimated total cost.

Algorithm / Steps: Initialize V1; calculate f; repeatedly expand the lowest-f node; update successors until V6 is reached.

Program:

```
import heapq

graph = {
    "V1": [("V2", 2), ("V3", 4)],
    "V2": [("V4", 1)],
    "V3": [("V4", 2)],
    "V4": [("V5", 3)],
    "V5": [("V6", 2)],
    "V6": []
}
h = {"V1":7,"V2":6,"V3":3,"V4":2,"V5":1,"V6":0}

def astar(start, goal):
    pq = [(h[start], 0, start, [start])]
    best = {start: 0}
    while pq:
        f,g,u,path = heapq.heappop(pq)
        if u == goal:
            return path,g
        for v,c in graph[u]:
            ng = g+c
            if ng < best.get(v, float("inf")):
                best[v]=ng
                heapq.heappush(pq,(ng+h[v],ng,v,path+[v]))
    return None, float("inf")

path,cost=astar("V1","V6")
print("Path:", " -> ".join(path))
print("Cost:", cost)
```

Output: Path: V1 -> V2 -> V4 -> V5 -> V6; total cost is printed.

Result: A* finds a shortest path from V1 to V6.

Q2) Solve Map Coloring using CSP.

Aim: To color adjacent regions differently using backtracking.

Theory: Variables are vertices, domains are available colors, and constraints prohibit equal colors on adjacent vertices.

Algorithm / Steps: Try colors recursively and backtrack on conflicts.

Program:

```
def is_safe(v, graph, colors, c):
    for u in graph[v]:
        if colors[u] == c:
            return False
    return True

def color_graph(graph, m):
    colors = [-1] * len(graph)

    def backtrack(v):
```

```
        if v == len(graph):
            return True
        for c in range(m):
            if is_safe(v, graph, colors, c):
                colors[v] = c
                if backtrack(v + 1):
                    return True
                colors[v] = -1
        return False

    if backtrack(0):
        return colors
    return None

# A-B-C-D graph
graph = [[1,2], [0,2,3], [0,1,3], [1,2]]
solution = color_graph(graph, 3)
print("Color assignment:", solution)
```

Output: A valid color assignment is printed.

Result: Map coloring is solved as a CSP.

OR — Alternative Q2

Q2 (OR)) Develop Tic-Tac-Toe using Minimax.

Aim: To create a small game where the computer selects an optimal move.

Theory: Minimax treats the computer as MAX and the opponent as MIN and recursively evaluates terminal game states.

Algorithm / Steps: Generate legal moves; apply a move; recursively evaluate; choose maximum for computer and minimum for opponent.

Program:

```
# Compact Tic-Tac-Toe Minimax
import math

def winner(b):
    lines=[(0,1,2),(3,4,5),(6,7,8),(0,3,6),(1,4,7),(2,5,8),(0,4,8),(2,4,6)]
    for a,c,d in lines:
        if b[a] and b[a]==b[c]==b[d]: return b[a]
    return None

def minimax(b, maximizing):
    w=winner(b)
    if w=="X": return 1
    if w=="O": return -1
    if all(b): return 0
    vals=[]
    for i in range(9):
        if not b[i]:
            b[i]="X" if maximizing else "O"
            vals.append(minimax(b,not maximizing))
            b[i]=""
    return max(vals) if maximizing else min(vals)

board=[""]*9
best=-math.inf; move=None
for i in range(9):
    board[i]="X"; score=minimax(board,False); board[i]=""
    if score>best: best,move=score,i
print("Computer best move:",move)
```

Output: The best move index is printed.

Result: Minimax selects the move with the best game-theoretic value.

Q3) Viva: Prepare definitions, working steps, key terms, complexity/accuracy, and one practical example for the programs above.

SLIP 8 — CS-321-VSC-P PRACTICAL

Foundation of Artificial Intelligence and Machine Learning

Q1) Solve the 4-Puzzle Problem using A*.

Aim: To find a path from a start 2×2 puzzle state to the goal state using A*.

Theory: A* uses a priority queue and heuristic. For the 4-puzzle, the misplaced-tile count is used.

Algorithm / Steps: Represent a state as a tuple; generate moves of the blank tile; compute f=g+h; continue until the goal.

Program:

```
import heapq

# State: (1,2,3,0), goal: (1,2,3,0)
# Change start_state to test another 4-puzzle arrangement.
start_state = (1,0,3,2)
goal = (1,2,3,0)

def h(s):
    return sum(s[i] != goal[i] and s[i] != 0 for i in range(4))

def neighbors(s):
    z=s.index(0)
    r,c=divmod(z,2)
    for dr,dc in [(1,0),(-1,0),(0,1),(0,-1)]:
        nr,nc=r+dr,c+dc
        if 0<=nr<2 and 0<=nc<2:
            j=nr*2+nc
            a=list(s); a[z],a[j]=a[j],a[z]
            yield tuple(a)

def astar(start):
    pq=[(h(start),0,start,[start])]
    seen={start:0}
    while pq:
        f,g,s,path=heapq.heappop(pq)
        if s==goal: return path
        for ns in neighbors(s):
            ng=g+1
            if ng<seen.get(ns,999):
                seen[ns]=ng
                heapq.heappush(pq,(ng+h(ns),ng,ns,path+[ns]))
    return None

path=astar(start_state)
for s in path:
    print(s)
```

Output: The sequence of puzzle states is printed.

Result: A* searches for a low-cost 4-puzzle solution.

Q2) Compare Minimax and Alpha-Beta on the same game tree.

Aim: To show that Alpha-Beta returns the same minimax value while reducing explored branches.

Theory: Minimax searches a complete game tree and chooses the maximum score for MAX and minimum score for MIN. Alpha-Beta pruning gives the same minimax decision while skipping branches that cannot affect the final choice. Best-case Alpha-Beta can reduce the effective search substantially with good move ordering; worst-case time remains O(b^d), while the stored game tree is O(b^d) for a straightforward implementation.

Algorithm / Steps: Evaluate the same leaf values with both algorithms and compare the returned value.

Program:

```
def minimax(depth, node, maximizing, values):
    if depth == 2:
        return values[node]
    if maximizing:
        return max(minimax(depth+1, 2*node, False, values),
                   minimax(depth+1, 2*node+1, False, values))
    return min(minimax(depth+1, 2*node, True, values),
               minimax(depth+1, 2*node+1, True, values))

def alphabeta(depth, node, maximizing, values, alpha, beta):
    if depth == 2:
        return values[node]
    if maximizing:
        best = float("-inf")
        for child in (2*node, 2*node+1):
            best=max(best, alphabeta(depth+1,child,False,values,alpha,beta))
            alpha=max(alpha,best)
            if beta<=alpha: break
        return best
    best=float("inf")
    for child in (2*node,2*node+1):
        best=min(best, alphabeta(depth+1,child,True,values,alpha,beta))
        beta=min(beta,best)
        if beta<=alpha: break
    return best

values=[3,5,2,9]
print("Minimax:", minimax(0,0,True,values))
print("Alpha-Beta:", alphabeta(0,0,True,values,float("-inf"),float("inf")))
```

Output: Both algorithms return the same optimal value.

Result: Alpha-Beta preserves the minimax result while enabling pruning.

OR — Alternative Q2

Q2 (OR)) Design a Rule-Based Expert System.

Aim: To make a simple decision using IF-THEN style rules.

Theory: A rule-based system contains facts and rules. The inference process applies rules whose conditions are satisfied.

Algorithm / Steps: Read input; test rule conditions; output the corresponding recommendation.

Program:

```
def expert_system(weather, temperature):
    if weather == "rainy":
        return "Carry an umbrella."
    if temperature > 35:
        return "Stay hydrated and avoid direct heat."
    return "Weather is suitable for normal outdoor activity."

weather = input("Enter weather (rainy/sunny): ").lower()
temperature = float(input("Enter temperature: "))
print("Advice:", expert_system(weather, temperature))
```

Output: Advice based on entered weather and temperature is displayed.

Result: The rule-based decision is produced from the supplied facts.

Q3) Viva: Prepare definitions, working steps, key terms, complexity/accuracy, and one practical example for the programs above.

SLIP 9 — CS-321-VSC-P PRACTICAL

Foundation of Artificial Intelligence and Machine Learning

Q1) Demonstrate Hill Climbing for maximum value.

Aim: To find a local maximum of a simple objective function.

Theory: Hill climbing repeatedly moves to a neighboring state with a better objective value and stops when no improvement exists.

Algorithm / Steps: Start at x=0; compare f(x-step) and f(x+step); move to the better neighbor; stop at a local maximum.

Program:

```
def f(x):
    return -x*x + 4*x

x = 0
step = 0.1

while True:
    current = f(x)
    left = f(x-step)
    right = f(x+step)
    if left > current:
        x -= step
    elif right > current:
        x += step
    else:
        break

print("Maximum x:", round(x,2))
print("Maximum value:", round(f(x),2))
```

Output: Maximum x is approximately 2.0 and maximum value approximately 4.0.

Result: Hill climbing reaches the maximum of the supplied quadratic.

Q2) Implement Forward Chaining.

Aim: To derive new facts repeatedly from known facts and rules.

Theory: Forward chaining is data-driven inference: begin with known facts and fire rules whose premises are satisfied.

Algorithm / Steps: Initialize facts; scan rules; add conclusions whose premises are present; repeat until no new facts appear.

Program:

```
facts = {"fever", "cough"}
rules = [
    ({"fever", "cough"}, "flu"),
    ({"flu"}, "take_rest"),
    ({"flu", "body_pain"}, "consult_doctor")
]

changed = True
while changed:
    changed = False
    for premises, conclusion in rules:
        if premises.issubset(facts) and conclusion not in facts:
            facts.add(conclusion)
            changed = True

print("Derived facts:", sorted(facts))
```

Output: Derived facts include fever, cough, flu and take_rest.

Result: Forward chaining derives all reachable conclusions.

OR — Alternative Q2

Q2 (OR)) Implement Decision Tree classifier and tune entropy/max_depth.

Aim: To train and evaluate a Decision Tree on the supplied diabetes dataset.

Theory: A decision tree recursively splits data. Here criterion='entropy' is tested with max_depth 2, 3 and 4; accuracy_score evaluates performance.

Algorithm / Steps: Create DataFrame; split 70:30; train entropy trees at depths 2–4; calculate accuracy; select best depth.

Program:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

data = {
"Pregnancies":[6,1,8,1,0,5,3,10,2,8,4],
"Glucose":[148,85,183,89,137,116,78,115,197,125,110],
"BP":[72,66,64,66,40,74,50,0,70,96,92],
"SkinThick":[35,29,0,23,35,0,32,0,45,0,0],
"Insulin":[0,0,0,94,168,0,88,0,543,0,0],
"BMI":[33.6,26.6,23.3,28.1,43.1,25.6,31,35.3,30.5,0,37.6],
"DiabetesPF":[0.627,0.351,0.672,0.167,2.288,0.201,0.248,0.134,0.158,0.232,0.191],
"Age":[50,31,32,21,33,30,26,29,53,54,30],
"Outcome":[1,0,1,0,1,0,1,0,1,1,0]
}
df=pd.DataFrame(data)
X=df.drop("Outcome",axis=1)
y=df["Outcome"]

Xtr,Xte,ytr,yte=train_test_split(X,y,test_size=0.30,random_state=42,stratify=y)
best=(0,None)
for depth in [2,3,4]:
    model=DecisionTreeClassifier(criterion="entropy",max_depth=depth,random_state=42)
    model.fit(Xtr,ytr)
    acc=accuracy_score(yte,model.predict(Xte))
    print("max_depth =",depth,"accuracy =",acc)
    if acc>best[0]: best=(acc,depth)
print("Best accuracy:",best[0],"Best max_depth:",best[1])
```

Output: Accuracy for each depth and the best depth are printed.

Result: The best tested Decision Tree configuration is reported.

Q3) Viva: Prepare definitions, working steps, key terms, complexity/accuracy, and one practical example for the programs above.

SLIP 10 — CS-321-VSC-P PRACTICAL

Foundation of Artificial Intelligence and Machine Learning

Q1) Implement Means-End Analysis for a goal-based problem.

Aim: To reduce the difference between a current state and a goal state.

Theory: Means-End Analysis repeatedly compares current and goal states, selects an operator that reduces the difference, and applies it.

Algorithm / Steps: Start at 2; compare with goal 7; move one step toward the goal until equal.

Program:

```
def means_end(start, goal):
    state = start
    while state != goal:
        if state < goal:
            state += 1
        else:
            state -= 1
        print("Current state:", state)
    print("Goal reached:", goal)

means_end(2, 7)
```

Output: Current state progresses to 7; Goal reached: 7.

Result: The goal is reached by repeatedly reducing the state difference.

Q2) Implement Random Forest classifier for classification tasks.

Aim: To classify student result using Study_Hours, Attendance and Assignment_Score.

Theory: Random Forest combines multiple decision trees and aggregates their predictions to improve robustness.

Algorithm / Steps: Create DataFrame; encode Result; split 70:30; fit RandomForestClassifier; calculate accuracy.

Program:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
import pandas as pd

data = {
"Study_Hours":[2,3,4,5,6,7,1,2,5,8],
"Attendance":[60,65,70,75,80,85,55,62,78,90],
"Assignment_Score":[55,60,65,70,75,80,50,58,72,85],
"Result":["Fail","Fail","Pass","Pass","Pass","Pass","Fail","Fail","Pass","Pass"]
}
df=pd.DataFrame(data)
le=LabelEncoder()
y=le.fit_transform(df["Result"])
X=df.drop("Result",axis=1)
Xtr,Xte,ytr,yte=train_test_split(X,y,test_size=.30,random_state=42)
model=RandomForestClassifier(random_state=42)
model.fit(Xtr,ytr)
print("Accuracy:",accuracy_score(yte,model.predict(Xte)))
```

Output: Accuracy is printed.

Result: The Random Forest classifier is trained and evaluated.

OR — Alternative Q2

Q2 (OR)) Demonstrate hyperplane classification using Linear SVM.

Aim: To classify supplied Iris data using StandardScaler and LinearSVC.

Theory: A linear SVM learns a separating hyperplane. Standardization puts features on comparable scales.

Algorithm / Steps: Build a Pipeline with StandardScaler and LinearSVC(C=1, loss='hinge'); fit and print training accuracy.

Program:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC

X=[[5.1,3.5,1.4,0.2],[4.9,3.0,1.4,0.2],
   [4.7,3.2,1.3,0.2],[4.6,3.1,1.5,0.2],
   [5.0,3.6,1.4,0.2],[7.0,3.2,4.7,1.4],
   [6.4,3.2,4.5,1.5],[6.9,3.1,4.9,1.5],
   [5.5,2.3,4.0,1.3],[6.5,2.8,4.6,1.5]]
y=[0,0,0,0,0,1,1,1,2,2]

model=Pipeline([
    ("scaler",StandardScaler()),
    ("svm",LinearSVC(C=1,loss="hinge",random_state=42))
])
model.fit(X,y)
print("Training accuracy:",model.score(X,y))
```

Output: Training accuracy is printed.

Result: Hyperplane classification is demonstrated using the required pipeline.

Q3) Viva: Prepare definitions, working steps, key terms, complexity/accuracy, and one practical example for the programs above.

SLIP 11 — CS-321-VSC-P PRACTICAL

Foundation of Artificial Intelligence and Machine Learning

Q1) Compare Minimax and Alpha-Beta on the same game tree.

Aim: To compare the decision value produced by Minimax and Alpha-Beta.

Theory: Minimax searches a complete game tree and chooses the maximum score for MAX and minimum score for MIN. Alpha-Beta pruning gives the same minimax decision while skipping branches that cannot affect the final choice. Best-case Alpha-Beta can reduce the effective search substantially with good move ordering; worst-case time remains O(b^d), while the stored game tree is O(b^d) for a straightforward implementation.

Algorithm / Steps: Run both algorithms on identical leaf values and compare outputs.

Program:

```
def minimax(depth, node, maximizing, values):
    if depth == 2:
        return values[node]
    if maximizing:
        return max(minimax(depth+1, 2*node, False, values),
                   minimax(depth+1, 2*node+1, False, values))
    return min(minimax(depth+1, 2*node, True, values),
               minimax(depth+1, 2*node+1, True, values))

def alphabeta(depth, node, maximizing, values, alpha, beta):
    if depth == 2:
        return values[node]
    if maximizing:
        best = float("-inf")
        for child in (2*node, 2*node+1):
            best=max(best, alphabeta(depth+1,child,False,values,alpha,beta))
            alpha=max(alpha,best)
            if beta<=alpha: break
        return best
    best=float("inf")
    for child in (2*node,2*node+1):
        best=min(best, alphabeta(depth+1,child,True,values,alpha,beta))
        beta=min(beta,best)
        if beta<=alpha: break
    return best

values=[3,5,2,9]
print("Minimax:", minimax(0,0,True,values))
print("Alpha-Beta:", alphabeta(0,0,True,values,float("-inf"),float("inf")))
```

Output: Both values are identical for the same game tree.

Result: Alpha-Beta optimizes the search without changing the minimax result.

Q2) Implement Backward Chaining.

Aim: To prove a goal by recursively checking rules that can derive it.

Theory: Backward chaining is goal-driven: start with the query and work backward to required premises.

Algorithm / Steps: Set a goal; if it is a fact return true; otherwise find a rule whose conclusion is the goal and recursively prove its premise.

Program:

```
facts = {"bird", "can_fly"}
rules = [
    ("bird", "animal"),
    ("animal", "living"),
    ("can_fly", "moves")
```

```
]

def prove(goal, visited=None):
    if visited is None:
        visited=set()
    if goal in facts:
        return True
    if goal in visited:
        return False
    visited.add(goal)

    for premise, conclusion in rules:
        if conclusion == goal and prove(premise, visited):
            return True
    return False

for goal in ["animal", "living", "moves"]:
    print(goal, "=>", prove(goal))
```

Output: animal, living and moves are proved as True.

Result: Backward chaining proves reachable goals from the knowledge base.

OR — Alternative Q2

Q2 (OR)) Implement Random Forest on categorical car_evaluation data.

Aim: To encode categorical features and train a Random Forest classifier.

Theory: Categorical features must be encoded before fitting a scikit-learn Random Forest. The class column is the target.

Algorithm / Steps: Create the 10-row data; inspect value counts and missing values; encode X; split 67:33; fit RF; print accuracy.

Program:

```
import pandas as pd
from sklearn.preprocessing import OrdinalEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

data = [
["vhigh","vhigh","2","2","small","low","unacc"],
["vhigh","vhigh","2","2","small","med","unacc"],
["vhigh","vhigh","2","2","small","high","unacc"],
["vhigh","vhigh","2","2","med","low","unacc"],
["vhigh","vhigh","2","2","med","med","unacc"],
["vhigh","vhigh","2","2","med","high","unacc"],
["vhigh","vhigh","2","2","big","low","unacc"],
["vhigh","vhigh","2","2","big","med","unacc"],
["vhigh","vhigh","2","2","big","high","unacc"],
["vhigh","vhigh","2","4","small","low","unacc"]
]
cols=["buying","maint","doors","persons","lug_boot","safety","class"]
df=pd.DataFrame(data,columns=cols)
print("Missing values:\n",df.isnull().sum())
for c in cols[:-1]:
    print(c,":\n",df[c].value_counts())

X=df.drop("class",axis=1)
y=df["class"]
enc=OrdinalEncoder()
X=enc.fit_transform(X)
Xtr,Xte,ytr,yte=train_test_split(X,y,test_size=.33,random_state=42)
model=RandomForestClassifier(random_state=42)
model.fit(Xtr,ytr)
print("Accuracy:",accuracy_score(yte,model.predict(Xte)))
```

Output: Frequency counts, missing-value counts and accuracy are printed.

Result: The categorical data is encoded and classified with Random Forest.

Q3) Viva: Prepare definitions, working steps, key terms, complexity/accuracy, and one practical example for the programs above.

SLIP 12 — CS-321-VSC-P PRACTICAL

Foundation of Artificial Intelligence and Machine Learning

Q1) Implement BFS for Monkey Banana.

Aim: To traverse the Monkey-Banana state space with BFS.

Theory: BFS uses FIFO queue traversal and the state sequence specified in the lab book.

Algorithm / Steps: Queue initial state; visit and enqueue its successor until banana is obtained.

Program:

```
from collections import deque

states = {
    "Monkey at Door": "Monkey at Box",
    "Monkey at Box": "Box Under Banana",
    "Box Under Banana": "Monkey Climbs Box",
    "Monkey Climbs Box": "Monkey Gets Banana"
}

q = deque(["Monkey at Door"])
visited = set()

while q:
    state = q.popleft()
    if state in visited:
        continue
    visited.add(state)
    print(state)
    if state == "Monkey Gets Banana":
        print("Goal: Banana Obtained!")
        break
    if state in states:
        q.append(states[state])
```

Output: Goal: Banana Obtained!

Result: Monkey-Banana problem is solved using BFS.

Q2) Compare Linear SVM and Non-Linear SVM (RBF).

Aim: To compare linear and RBF-kernel SVM predictions on the supplied dataset.

Theory: Linear SVM learns a straight hyperplane. RBF SVM can model curved/non-linear boundaries.

Algorithm / Steps: Create X/Y; train linear and RBF SVMs; predict the same test point; compare results.

Program:

```
import numpy as np
from sklearn.svm import SVC

X=np.array([[1,2],[2,3],[3,1],[4,2],
            [5,5],[6,6],[7,4],[8,5]])
y=np.array([0,0,0,0,1,1,1,1])

linear=SVC(kernel="linear").fit(X,y)
rbf=SVC(kernel="rbf").fit(X,y)

print("Linear SVM prediction:",linear.predict([[4.5,3.5]])[0])
print("RBF SVM prediction:",rbf.predict([[4.5,3.5]])[0])
```

Output: Predictions from both kernels are displayed.

Result: The two kernels are compared on identical data.

OR — Alternative Q2

Q2 (OR)) Implement ANN for Linear Regression.

Aim: To learn a continuous target from a numeric input using an ANN regressor.

Theory: An ANN regressor predicts continuous values. Mean Squared Error measures the difference between actual and predicted values.

Algorithm / Steps: Create X and y; train MLPRegressor; predict; calculate MSE.

Program:

```
import numpy as np
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_squared_error

X=np.array([[1],[2],[3],[4],[5]],dtype=float)
y=np.array([2,4,6,8,10],dtype=float)

model=MLPRegressor(hidden_layer_sizes=(5,),learning_rate_init=.01,
                   max_iter=2000,random_state=42)
model.fit(X,y)
pred=model.predict(X)
print("Predicted:",np.round(pred,2))
print("MSE:",mean_squared_error(y,pred))
```

Output: Predicted values and MSE are printed.

Result: ANN regression is implemented and evaluated with MSE.

Q3) Viva: Prepare definitions, working steps, key terms, complexity/accuracy, and one practical example for the programs above.

SLIP 13 — CS-321-VSC-P PRACTICAL

Foundation of Artificial Intelligence and Machine Learning

Q1) Solve Water Jug using BFS.

Aim: To obtain 2 litres from 4L and 3L jugs.

Theory: Water-jug states are represented as ordered pairs; BFS explores legal fill, empty and pour transitions.

Algorithm / Steps: Use queue and visited states; generate all legal transitions; stop when one jug contains 2 litres.

Program:

```
from collections import deque

def water_jug():
    capacity = (4, 3)
    q = deque([(0, 0)])
    parent = {(0, 0): None}
    action = {}

    while q:
        a, b = q.popleft()
        if a == 2 or b == 2:
            path = []
            cur = (a, b)
            while parent[cur] is not None:
                path.append((cur, action[cur]))
                cur = parent[cur]
            path.append(((0, 0), "Initial"))
            for state, act in reversed(path):
                print(act, "->", state)
            return

        next_states = [
            ((4, b), "Fill A"), ((a, 3), "Fill B"),
            ((0, b), "Empty A"), ((a, 0), "Empty B")
        ]
        t = min(a, 3-b)
        next_states.append(((a-t, b+t), "Pour A -> B"))
        t = min(b, 4-a)
        next_states.append(((a+t, b-t), "Pour B -> A"))

        for state, act in next_states:
            if state not in parent:
                parent[state] = (a, b)
                action[state] = act
                q.append(state)

water_jug()
```

Output: A solution state containing 2 litres is printed.

Result: Water Jug is solved using BFS.

Q2) Implement a Voting Classifier combining Logistic Regression and Decision Tree.

Aim: To combine two supervised models using hard voting.

Theory: A VotingClassifier aggregates predictions from base estimators. Hard voting selects the class receiving the majority of votes.

Algorithm / Steps: Load Iris; split data; create Logistic Regression and Decision Tree; combine with VotingClassifier; calculate accuracy.

Program:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

X,y=load_iris(return_X_y=True)
Xtr,Xte,ytr,yte=train_test_split(X,y,test_size=.30,random_state=42,stratify=y)

lr=LogisticRegression(max_iter=1000)
dt=DecisionTreeClassifier(random_state=42)
model=VotingClassifier([("lr",lr),("dt",dt)],voting="hard")
model.fit(Xtr,ytr)
print("Voting Classifier Accuracy:",accuracy_score(yte,model.predict(Xte)))
```

Output: Voting Classifier Accuracy is printed.

Result: The combined classifier is trained and evaluated.

OR — Alternative Q2

Q2 (OR)) Develop a simple Knowledge Graph.

Aim: To represent relationships among entities using triples.

Theory: A knowledge graph can be represented as subject-relation-object triples.

Algorithm / Steps: Create triples; iterate through them; retrieve relations for a selected entity.

Program:

```
facts=[
    ("Alice","studies","Computer Science"),
    ("Bob","works_at","ABC Company"),
    ("Alice","friends_with","Bob")
]

for s,r,o in facts:
    print(s, "--", r, "-->", o)

person="Alice"
print("Relations of",person,":")
for s,r,o in facts:
    if s==person:
        print(r,"->",o)
```

Output: Triples and Alice's relations are displayed.

Result: Entity relationships are represented as a simple knowledge graph.

Q3) Viva: Prepare definitions, working steps, key terms, complexity/accuracy, and one practical example for the programs above.

SLIP 14 — CS-321-VSC-P PRACTICAL

Foundation of Artificial Intelligence and Machine Learning

Q1) Implement BFS traversal for a graph.

Aim: To visit all reachable graph vertices level-wise.

Theory: BFS uses a queue and visited set and is suitable for unweighted shortest-path exploration.

Algorithm / Steps: Initialize queue; pop vertex; visit neighbors; enqueue unvisited neighbors.

Program:

```
from collections import deque

graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [], "E": [], "F": []
}

def bfs(start):
    q = deque([start])
    visited = {start}
    order = []

    while q:
        v = q.popleft()
        order.append(v)
        for u in graph[v]:
            if u not in visited:
                visited.add(u)
                q.append(u)
    return order

print("BFS Traversal:", bfs("A"))
```

Output: A B C D E F

Result: BFS traversal is completed.

Q2) Compare Supervised and Unsupervised Learning.

Aim: To distinguish labelled-data learning from structure discovery in unlabelled data.

Theory: Supervised learning uses labelled data (input + target) to learn a mapping. Typical tasks are classification and regression. Unsupervised learning uses unlabelled data to discover structure, such as clusters. Examples: Logistic Regression, Decision Tree and SVM are supervised; K-Means and dimensionality reduction are unsupervised. Evaluation commonly uses accuracy/F1 for classification and MSE for regression; clustering can use inertia, silhouette score and Davies-Bouldin index.

Algorithm / Steps: List data requirement, task type, examples and common evaluation measures; compare them point by point.

Output: Supervised: labelled data, classification/regression. Unsupervised: unlabelled data, clustering/structure discovery.

Result: The two learning paradigms are compared.

OR — Alternative Q2

Q2 (OR)) Demonstrate basic Ontological Engineering.

Aim: To represent classes, subclasses, properties and relationships.

Theory: An ontology organizes domain concepts into classes and relations. Subclasses inherit the meaning of their parent class.

Algorithm / Steps: Define parent classes and subclasses; define relationships/properties; print the ontology structure.

Program:

```
classes={
    "Animal": ["Dog","Cat"],
    "Vehicle": ["Car","Bike"]
}
relationships=[
    ("Dog","is_a","Animal"),
    ("Car","is_a","Vehicle"),
    ("Dog","has_property","Four legs")
]

for parent,children in classes.items():
    print(parent,":",children)
for rel in relationships:
    print(rel[0],rel[1],rel[2])
```

Output: Classes and relationships are displayed.

Result: Basic ontology concepts are represented programmatically.

Q3) Viva: Prepare definitions, working steps, key terms, complexity/accuracy, and one practical example for the programs above.

SLIP 15 — CS-321-VSC-P PRACTICAL

Foundation of Artificial Intelligence and Machine Learning

Q1) Solve a state-space search problem using DFS, Start vertex=1.

Aim: To search a state space depth-first starting from vertex 1.

Theory: DFS explores one branch deeply before backtracking.

Algorithm / Steps: Start at 1; mark visited; recursively visit unvisited successors.

Program:

```
graph={
    1:[2,3],
    2:[4],
    3:[5],
    4:[],5:[]
}

def dfs(node,visited):
    visited.add(node)
    print(node,end=" ")
    for nxt in graph[node]:
        if nxt not in visited:
            dfs(nxt,visited)

print("DFS State-Space Traversal:")
dfs(1,set())
```

Output: DFS State-Space Traversal: 1 2 4 3 5

Result: The state space is traversed using DFS.

Q2) Demonstrate clustering objective functions: any two.

Aim: To calculate and plot clustering quality measures across K.

Theory: K-Means inertia is within-cluster sum of squares. Silhouette measures cohesion/separation; Davies-Bouldin measures cluster similarity and lower is generally better.

Algorithm / Steps: Fit K-Means for K=2..7; calculate inertia, silhouette and Davies-Bouldin; plot each metric against K.

Program:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, davies_bouldin_score

X,_=make_blobs(n_samples=300,centers=3,cluster_std=1.2,random_state=42)

ks=range(2,8)
inertia=[]
sil=[]
db=[]

for k in ks:
    model=KMeans(n_clusters=k,random_state=42,n_init=10).fit(X)
    inertia.append(model.inertia_)
    sil.append(silhouette_score(X,model.labels_))
    db.append(davies_bouldin_score(X,model.labels_))

plt.figure()
plt.plot(list(ks),inertia,marker="o")
plt.xlabel("K"); plt.ylabel("Inertia (WCSS)"); plt.title("Elbow Curve")
```

```
plt.show()

plt.figure()
plt.plot(list(ks),sil,marker="o")
plt.xlabel("K"); plt.ylabel("Silhouette Score"); plt.show()

plt.figure()
plt.plot(list(ks),db,marker="o")
plt.xlabel("K"); plt.ylabel("Davies-Bouldin Index"); plt.show()
```

Output: Three plots are generated: Elbow/WCSS, Silhouette, and Davies-Bouldin.

Result: Clustering objective functions are demonstrated across K.

OR — Alternative Q2

Q2 (OR)) Build a Knowledge Base using logical rules and facts.

Aim: To store facts and infer a new fact from rules.

Theory: A knowledge base contains facts and logical rules. A rule can derive a conclusion when all its premises are present.

Algorithm / Steps: Initialize facts and rules; check rule premises; add satisfied conclusions.

Program:

```
facts={"bird","has_wings"}
rules=[
    ({"bird","has_wings"},"can_fly"),
    ({"can_fly"},"moves_fast")
]

print("Facts:",facts)
for premises,conclusion in rules:
    if premises.issubset(facts):
        facts.add(conclusion)
print("Knowledge Base after inference:",facts)
```

Output: Knowledge Base after inference contains can_fly and moves_fast.

Result: A simple rule-based knowledge base is constructed.

Q3) Viva: Prepare definitions, working steps, key terms, complexity/accuracy, and one practical example for the programs above.

SLIP 16 — CS-321-VSC-P PRACTICAL

Foundation of Artificial Intelligence and Machine Learning

Q1) Implement Depth Limited Search.

Aim: To perform DFS with a fixed depth limit.

Theory: DLS prevents expansion beyond a specified depth and avoids unlimited DFS paths.

Algorithm / Steps: Recursively search; decrement limit; return success at goal; stop at limit zero.

Program:

```
graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F", "G"],
    "D": [], "E": [], "F": [], "G": []
}

def dls(node, goal, limit, path):
    path.append(node)
    if node == goal:
        return True
    if limit == 0:
        path.pop()
        return False
    for child in graph[node]:
        if dls(child, goal, limit - 1, path):
            return True
    path.pop()
    return False

path = []
found = dls("A", "G", 2, path)
print("Path:", path if found else "Goal not found")
```

Output: Path to G is printed if the goal is within the limit.

Result: DLS is implemented.

Q2) Implement Backward Chaining.

Aim: To prove goals by recursively checking supporting premises.

Theory: Backward chaining starts from a query and searches for rules that can conclude it.

Algorithm / Steps: Check if goal is fact; otherwise select a rule concluding the goal and recursively prove its premise.

Program:

```
facts = {"bird", "can_fly"}
rules = [
    ("bird", "animal"),
    ("animal", "living"),
    ("can_fly", "moves")
]

def prove(goal, visited=None):
    if visited is None:
        visited=set()
    if goal in facts:
        return True
    if goal in visited:
        return False
    visited.add(goal)
```

```
    for premise, conclusion in rules:
        if conclusion == goal and prove(premise, visited):
            return True
    return False

for goal in ["animal", "living", "moves"]:
    print(goal, "=>", prove(goal))
```

Output: Derived goals are shown as True.

Result: Backward chaining inference is completed.

OR — Alternative Q2

Q2 (OR)) Implement K-Means on country_data.csv with elbow method.

Aim: To cluster countries after standardizing numeric development indicators.

Theory: K-Means partitions observations into K clusters by minimizing within-cluster squared distance. StandardScaler prevents large-scale features from dominating.

Algorithm / Steps: Read country_data.csv; select numeric columns excluding country name; standardize; compute K-Means inertia for K=1..10; plot elbow; use K=3 as requested.

Program:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

df=pd.read_csv("country_data.csv")
cols=["child_mortality","exports","health","imports","income",
      "inflation","life_expectancy","total_fer","gdp"]
X=StandardScaler().fit_transform(df[cols])

inertia=[]
for k in range(1,11):
    model=KMeans(n_clusters=k,random_state=42,n_init=10).fit(X)
    inertia.append(model.inertia_)

plt.plot(range(1,11),inertia,marker="o")
plt.xlabel("K"); plt.ylabel("Inertia"); plt.title("Elbow Method")
plt.show()

model=KMeans(n_clusters=3,random_state=42,n_init=10)
labels=model.fit_predict(X)
print("Cluster labels:",labels)
```

Output: Elbow curve and cluster labels are displayed.

Result: Country data is normalized and clustered using K-Means.

Q3) Viva: Prepare definitions, working steps, key terms, complexity/accuracy, and one practical example for the programs above.

SLIP 17 — CS-321-VSC-P PRACTICAL

Foundation of Artificial Intelligence and Machine Learning

Q1) Implement Best First Search using heuristic values.

Aim: To reach a goal by repeatedly selecting the node with the lowest heuristic.

Theory: Best First Search is an informed search. It uses h(n) to prioritize promising nodes.

Algorithm / Steps: Initialize priority queue with start; pop smallest h; if goal stop; enqueue unvisited successors with their heuristic.

Program:

```
from queue import PriorityQueue

graph={"A":["B","C"],"B":["D","E"],"C":["F","G"],
       "D":[],"E":[],"F":[],"G":[]}
h={"A":10,"B":8,"C":5,"D":7,"E":6,"F":2,"G":0}

pq=PriorityQueue()
pq.put((h["A"],"A"))
visited=set()
while not pq.empty():
    _,u=pq.get()
    if u in visited: continue
    visited.add(u)
    print(u,end=" ")
    if u=="G":
        print("\nGoal Found!")
        break
    for v in graph[u]:
        if v not in visited:
            pq.put((h[v],v))
```

Output: Traversal follows A C G and reaches G.

Result: Best First Search reaches the goal using heuristic priority.

Q2) Implement Propositional Logic using AND, OR and IMPLIES.

Aim: To evaluate propositional expressions with the requested operators.

Theory: Propositions have Boolean values; implication P→Q is equivalent to ¬P OR Q.

Algorithm / Steps: Set P and Q; evaluate AND, OR and implication; print results.

Program:

```
P=True
Q=False
print("P AND Q =",P and Q)
print("P OR Q =",P or Q)
print("P IMPLIES Q =",(not P) or Q)
```

Output: Boolean results are printed.

Result: The required propositional operators are evaluated.

OR — Alternative Q2

Q2 (OR)) Calculate Euclidean distance between two input vectors.

Aim: To calculate Euclidean distance between two numeric vectors.

Theory: For vectors x and y, distance = sqrt(sum((xi-yi)^2)).

Algorithm / Steps: Read two vectors; subtract corresponding values; square and sum; take square root.

Program:

```
import math

a=list(map(float,input("Enter first vector: ").split()))
b=list(map(float,input("Enter second vector: ").split()))

distance=math.sqrt(sum((x-y)**2 for x,y in zip(a,b)))
print("Data Point 1:",a)
print("Data Point 2:",b)
print("Euclidean Distance =",round(distance,3))
```

Output: For [2,3,4] and [5,7,6], Euclidean Distance = 5.099.

Result: Euclidean distance is calculated successfully.

Q3) Viva: Prepare definitions, working steps, key terms, complexity/accuracy, and one practical example for the programs above.

SLIP 18 — CS-321-VSC-P PRACTICAL

Foundation of Artificial Intelligence and Machine Learning

Q1) Implement A* shortest-path search.

Aim: To find a shortest path using g(n), h(n) and f(n)=g(n)+h(n).

Theory: A* expands the node with the smallest estimated total cost f(n).

Algorithm / Steps: Initialize priority queue; expand lowest-f node; update costs; stop at goal.

Program:

```
import heapq

graph = {
    "A": [("B", 2), ("C", 4)],
    "B": [("D", 1)],
    "C": [("D", 2)],
    "D": [("E", 3)],
    "E": [("F", 2)],
    "F": []
}
h = {"A": 7, "B": 6, "C": 3, "D": 2, "E": 1, "F": 0}

def a_star(start, goal):
    pq = [(h[start], 0, start, [start])]
    best_g = {start: 0}

    while pq:
        f, g, node, path = heapq.heappop(pq)
        if node == goal:
            return path, g
        for nxt, cost in graph[node]:
            new_g = g + cost
            if new_g < best_g.get(nxt, float("inf")):
                best_g[nxt] = new_g
                heapq.heappush(pq, (new_g + h[nxt], new_g,
                                    nxt, path + [nxt]))
    return None, float("inf")

path, cost = a_star("A", "F")
print("Shortest Path:", " -> ".join(path))
print("Path Cost:", cost)
```

Output: Shortest path and cost are printed.

Result: A* finds a shortest path.

Q2) Implement ANN for Linear Regression.

Aim: To predict continuous outputs using an ANN regressor.

Theory: MLPRegressor learns a non-linear function and can approximate a linear relationship.

Algorithm / Steps: Train on numeric X/y; predict; evaluate using MSE.

Program:

```
import numpy as np
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_squared_error

X=np.array([[1],[2],[3],[4],[5]],dtype=float)
y=np.array([2,4,6,8,10],dtype=float)

model=MLPRegressor(hidden_layer_sizes=(5,),learning_rate_init=.01,
```

```
                    max_iter=2000,random_state=42)
model.fit(X,y)
pred=model.predict(X)
print("Predicted:",np.round(pred,2))
print("MSE:",mean_squared_error(y,pred))
```

Output: Predictions and MSE are printed.

Result: ANN linear regression is demonstrated.

OR — Alternative Q2

Q2 (OR)) Visualize SVM decision boundaries.

Aim: To visualize the classification regions created by an SVM.

Theory: A classifier partitions feature space into decision regions. For two features, predictions can be evaluated on a grid and displayed as a contour plot.

Algorithm / Steps: Train SVC on 2D data; generate a mesh grid; predict every grid point; contour-plot regions and scatter training data.

Program:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVC

X=np.array([[1,2],[2,3],[3,1],[4,2],[5,5],[6,6],[7,4],[8,5]])
y=np.array([0,0,0,0,1,1,1,1])
model=SVC(kernel="rbf").fit(X,y)

x1=np.linspace(0,9,200)
x2=np.linspace(0,7,200)
xx,yy=np.meshgrid(x1,x2)
Z=model.predict(np.c_[xx.ravel(),yy.ravel()]).reshape(xx.shape)

plt.contourf(xx,yy,Z,alpha=.25)
plt.scatter(X[:,0],X[:,1],c=y)
plt.xlabel("Feature 1"); plt.ylabel("Feature 2")
plt.title("SVM Decision Boundary")
plt.show()
```

Output: A decision-boundary plot is displayed.

Result: The SVM decision regions are visualized.

Q3) Viva: Prepare definitions, working steps, key terms, complexity/accuracy, and one practical example for the programs above.

SLIP 19 — CS-321-VSC-P PRACTICAL

Foundation of Artificial Intelligence and Machine Learning

Q1) Solve 8-Puzzle using A*.

Aim: To solve the 8-puzzle using A* and a heuristic.

Theory: The 8-puzzle state is a 3×3 arrangement. A* uses path cost plus a heuristic such as Manhattan distance.

Algorithm / Steps: Represent board as a tuple; generate blank moves; calculate Manhattan distance; use priority queue; stop at goal.

Program:

```
import heapq

start=(1,2,3,4,0,6,7,5,8)
goal =(1,2,3,4,5,6,7,8,0)

def h(s):
    return sum(abs(i//3-goal.index(v)//3)+
               abs(i%3-goal.index(v)%3)
               for i,v in enumerate(s) if v)

def next_states(s):
    z=s.index(0); r,c=divmod(z,3)
    for dr,dc in [(1,0),(-1,0),(0,1),(0,-1)]:
        nr,nc=r+dr,c+dc
        if 0<=nr<3 and 0<=nc<3:
            j=nr*3+nc
            a=list(s); a[z],a[j]=a[j],a[z]
            yield tuple(a)

pq=[(h(start),0,start,[start])]
best={start:0}
while pq:
    f,g,s,path=heapq.heappop(pq)
    if s==goal:
        for state in path: print(state)
        break
    for ns in next_states(s):
        ng=g+1
        if ng<best.get(ns,999):
            best[ns]=ng
            heapq.heappush(pq,(ng+h(ns),ng,ns,path+[ns]))
```

Output: The sequence of 8-puzzle states is displayed.

Result: A* finds a solution path to the goal state.

Q2) Build a CNN for Orange vs Apple image recognition.

Aim: To classify images into Orange and Apple classes using a CNN.

Theory: CNNs use convolution and pooling to learn spatial image features. A binary sigmoid output is suitable for two classes.

Algorithm / Steps: Load directory images; resize; build convolution/pooling layers; train; evaluate.

Program:

```
# Basic CNN structure for Orange vs Apple image classification.
# Arrange images as:
# dataset/orange/..., dataset/apple/...

import tensorflow as tf
from tensorflow.keras import Sequential
```

```
from tensorflow.keras.layers import Conv2D,MaxPooling2D,Flatten,Dense

train=tf.keras.utils.image_dataset_from_directory(
    "dataset", image_size=(128,128), batch_size=32)

model=Sequential([
    Conv2D(32,(3,3),activation="relu",input_shape=(128,128,3)),
    MaxPooling2D(2,2),
    Conv2D(64,(3,3),activation="relu"),
    MaxPooling2D(2,2),
    Flatten(),
    Dense(64,activation="relu"),
    Dense(1,activation="sigmoid")
])
model.compile(optimizer="adam",loss="binary_crossentropy",
              metrics=["accuracy"])
model.fit(train,epochs=10)
print("Orange/Apple CNN training completed.")
```

Output: Training completes and accuracy is reported by Keras.

Result: A basic Orange-vs-Apple CNN is implemented.

OR — Alternative Q2

Q2 (OR)) Analyze SVM performance on a classification dataset.

Aim: To train an SVM and report its classification accuracy.

Theory: SVM performance can be evaluated using accuracy on held-out test data.

Algorithm / Steps: Split data; train SVC; predict test data; calculate accuracy and classification report.

Program:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score,classification_report

X,y=load_iris(return_X_y=True)
Xtr,Xte,ytr,yte=train_test_split(X,y,test_size=.30,random_state=42,stratify=y)
model=SVC(kernel="rbf").fit(Xtr,ytr)
pred=model.predict(Xte)
print("Accuracy:",accuracy_score(yte,pred))
print(classification_report(yte,pred))
```

Output: Accuracy and classification report are displayed.

Result: SVM performance is measured on the classification dataset.

Q3) Viva: Prepare definitions, working steps, key terms, complexity/accuracy, and one practical example for the programs above.

SLIP 20 — CS-321-VSC-P PRACTICAL

Foundation of Artificial Intelligence and Machine Learning

Q1) Demonstrate Hill Climbing for $F(x)=-x^2+4x$.

Aim: To find the maximum of the supplied objective function.

Theory: The quadratic reaches its maximum at x=2 with F(2)=4; hill climbing approaches the maximum by local improvement.

Algorithm / Steps: Start from x=0; compare nearby objective values; move uphill until no neighbor is better.

Program:

```
def f(x):
    return -x*x + 4*x

x = 0
step = 0.1

while True:
    current = f(x)
    left = f(x-step)
    right = f(x+step)
    if left > current:
        x -= step
    elif right > current:
        x += step
    else:
        break

print("Maximum x:", round(x,2))
print("Maximum value:", round(f(x),2))
```

Output: Maximum x ≈ 2.0; Maximum value ≈ 4.0.

Result: The maximum is found.

Q2) Implement CNN for basic object classification (person, car or dog).

Aim: To build a CNN that classifies images into three object classes.

Theory: A CNN learns image features through convolution and pooling, followed by dense classification layers.

Algorithm / Steps: Load three-class directory; build CNN; train with sparse categorical cross-entropy; evaluate.

Program:

```
# Basic CNN-style binary object classifier for person/car/dog.
# This classifies the image into one of the folders/classes.
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Conv2D,MaxPooling2D,Flatten,Dense

data=tf.keras.utils.image_dataset_from_directory(
    "objects",image_size=(128,128),batch_size=32)

model=Sequential([
    Conv2D(32,3,activation="relu",input_shape=(128,128,3)),
    MaxPooling2D(),
    Conv2D(64,3,activation="relu"),
    MaxPooling2D(),
    Flatten(),
    Dense(128,activation="relu"),
    Dense(3,activation="softmax")
])
model.compile(optimizer="adam",
```

```
              loss="sparse_categorical_crossentropy",metrics=["accuracy"])
model.fit(data,epochs=10)
print("Object classification model trained.")
```

Output: Training accuracy/loss is displayed.

Result: A three-class CNN object classifier is trained.

OR — Alternative Q2

Q2 (OR)) Compare Supervised and Unsupervised Learning.

Aim: To compare the two learning paradigms.

Theory: Supervised learning uses labelled data (input + target) to learn a mapping. Typical tasks are classification and regression. Unsupervised learning uses unlabelled data to discover structure, such as clusters. Examples: Logistic Regression, Decision Tree and SVM are supervised; K-Means and dimensionality reduction are unsupervised. Evaluation commonly uses accuracy/F1 for classification and MSE for regression; clustering can use inertia, silhouette score and Davies-Bouldin index.

Algorithm / Steps: Compare labels, objectives, examples and evaluation measures.

Output: Supervised uses labelled targets; unsupervised discovers structure without target labels.

Result: The paradigms are compared.

Q3) Viva: Prepare definitions, working steps, key terms, complexity/accuracy, and one practical example for the programs above.

SLIP 21 — CS-321-VSC-P PRACTICAL

Foundation of Artificial Intelligence and Machine Learning

Q1) Implement Iterative Deepening Search (IDS).

Aim: To combine the completeness of breadth expansion with depth-limited searches.

Theory: IDS repeatedly runs DLS with increasing depth limits until the goal is found.

Algorithm / Steps: Run DLS at depth 0, then 1, 2, ...; stop when goal is found.

Program:

```
graph={
    "A":["B","C"],
    "B":["D","E"],
    "C":["F"],
    "D":[],"E":[],"F":[]
}

def dls(node,goal,limit,path):
    path.append(node)
    if node==goal: return True
    if limit==0:
        path.pop(); return False
    for nxt in graph[node]:
        if dls(nxt,goal,limit-1,path): return True
    path.pop(); return False

for depth in range(5):
    path=[]
    if dls("A","F",depth,path):
        print("Goal found at depth",depth,":",path)
        break
```

Output: Goal found at the first depth where it is reachable.

Result: IDS finds the goal using increasing depth limits.

Q2) Implement Gaussian Naive Bayes.

Aim: To classify numerical data using GaussianNB.

Theory: GaussianNB assumes each feature follows a Gaussian distribution within each class.

Algorithm / Steps: Create numeric training data; fit GaussianNB; predict a test vector.

Program:

```
from sklearn.naive_bayes import GaussianNB

X=[[1.0,2.0],[1.2,1.8],[5.0,6.0],[5.2,5.8]]
y=[0,0,1,1]

model=GaussianNB().fit(X,y)
print("Prediction:",model.predict([[5.1,6.1]]))
```

Output: Prediction is displayed.

Result: Gaussian Naive Bayes classifies the numeric sample.

OR — Alternative Q2

Q2 (OR)) Solve Map Coloring using CSP.

Aim: To assign valid colors to graph vertices.

Theory: CSP uses variables, domains and constraints; backtracking searches valid assignments.

Algorithm / Steps: Assign colors recursively and backtrack on conflicts.

Program:

```
def is_safe(v, graph, colors, c):
    for u in graph[v]:
        if colors[u] == c:
            return False
    return True

def color_graph(graph, m):
    colors = [-1] * len(graph)

    def backtrack(v):
        if v == len(graph):
            return True
        for c in range(m):
            if is_safe(v, graph, colors, c):
                colors[v] = c
                if backtrack(v + 1):
                    return True
                colors[v] = -1
        return False

    if backtrack(0):
        return colors
    return None

# A-B-C-D graph
graph = [[1,2], [0,2,3], [0,1,3], [1,2]]
solution = color_graph(graph, 3)
print("Color assignment:", solution)
```

Output: Valid color assignment is printed.

Result: Map coloring is solved using CSP.

Q3) Viva: Prepare definitions, working steps, key terms, complexity/accuracy, and one practical example for the programs above.

SLIP 22 — CS-321-VSC-P PRACTICAL

Foundation of Artificial Intelligence and Machine Learning

Q1) Solve Tower of Hanoi using state-space representation.

Aim: To generate the sequence of legal moves for the Tower of Hanoi.

Theory: The problem can be represented as states of disk positions. The recursive solution decomposes the problem into smaller states.

Algorithm / Steps: Move n-1 disks to auxiliary; move largest to destination; move n-1 from auxiliary to destination.

Program:

```
def hanoi(n,source,aux,dest):
    if n==1:
        print(source,"->",dest)
        return
    hanoi(n-1,source,dest,aux)
    print(source,"->",dest)
    hanoi(n-1,aux,source,dest)

print("Tower of Hanoi (3 disks):")
hanoi(3,"A","B","C")
```

Output: A sequence of moves for 3 disks is displayed.

Result: Tower of Hanoi moves are generated.

Q2) Solve N-Queens using Backtracking.

Aim: To place N queens so that no two attack each other.

Theory: Each row is a variable; columns are domain values; row/column/diagonal conflicts are constraints.

Algorithm / Steps: Place a queen row by row; check column and diagonals; backtrack on conflict.

Program:

```
def solve_nqueens(n):
    board=[-1]*n
    solutions=[]

    def safe(row,col):
        for r in range(row):
            c=board[r]
            if c==col or abs(c-col)==abs(r-row):
                return False
        return True

    def backtrack(row):
        if row==n:
            solutions.append(board.copy())
            return
        for col in range(n):
            if safe(row,col):
                board[row]=col
                backtrack(row+1)
                board[row]=-1

    backtrack(0)
    return solutions

sol=solve_nqueens(4)
print("Number of solutions:",len(sol))
for row in sol[0]:
    print("."*row+"Q"+"."*(3-row))
```

Output: A 4-Queens solution and number of solutions are printed.

Result: N-Queens is solved by backtracking.

OR — Alternative Q2

Q2 (OR)) Design a Rule-Based Expert System.

Aim: To make a simple decision using rules and facts.

Theory: Rule-based systems apply IF-THEN knowledge to inputs.

Algorithm / Steps: Read conditions; match them to rules; print the corresponding conclusion.

Program:

```
def expert_system(weather, temperature):
    if weather == "rainy":
        return "Carry an umbrella."
    if temperature > 35:
        return "Stay hydrated and avoid direct heat."
    return "Weather is suitable for normal outdoor activity."

weather = input("Enter weather (rainy/sunny): ").lower()
temperature = float(input("Enter temperature: "))
print("Advice:", expert_system(weather, temperature))
```

Output: Advice is displayed.

Result: The expert system produces a rule-based recommendation.

Q3) Viva: Prepare definitions, working steps, key terms, complexity/accuracy, and one practical example for the programs above.

SLIP 23 — CS-321-VSC-P PRACTICAL

Foundation of Artificial Intelligence and Machine Learning

Q1) Demonstrate Hill Climbing for $F(x)=-x^2+4x$.

Aim: To find the maximum of the specified function.

Theory: Hill climbing improves the objective locally. For this function the maximum occurs at x=2.

Algorithm / Steps: Evaluate neighboring x values; move toward the higher value until no improvement remains.

Program:

```
def f(x):
    return -x*x + 4*x

x = 0
step = 0.1

while True:
    current = f(x)
    left = f(x-step)
    right = f(x+step)
    if left > current:
        x -= step
    elif right > current:
        x += step
    else:
        break

print("Maximum x:", round(x,2))
print("Maximum value:", round(f(x),2))
```

Output: x ≈ 2 and F(x) ≈ 4.

Result: The maximum is reached.

Q2) Solve Map Coloring using CSP.

Aim: To color adjacent graph vertices differently.

Theory: CSP defines variables, domains and constraints; backtracking searches valid color assignments.

Algorithm / Steps: Try each color; reject colors used by adjacent vertices; backtrack when needed.

Program:

```
def is_safe(v, graph, colors, c):
    for u in graph[v]:
        if colors[u] == c:
            return False
    return True

def color_graph(graph, m):
    colors = [-1] * len(graph)

    def backtrack(v):
        if v == len(graph):
            return True
        for c in range(m):
            if is_safe(v, graph, colors, c):
                colors[v] = c
                if backtrack(v + 1):
                    return True
                colors[v] = -1
        return False
```

```
    if backtrack(0):
        return colors
    return None

# A-B-C-D graph
graph = [[1,2], [0,2,3], [0,1,3], [1,2]]
solution = color_graph(graph, 3)
print("Color assignment:", solution)
```

Output: A valid color assignment is printed.

Result: Map coloring is solved.

OR — Alternative Q2

Q2 (OR)) Compare Supervised and Unsupervised Learning.

Aim: To compare labelled and unlabelled learning.

Theory: Supervised learning uses labelled data (input + target) to learn a mapping. Typical tasks are classification and regression. Unsupervised learning uses unlabelled data to discover structure, such as clusters. Examples: Logistic Regression, Decision Tree and SVM are supervised; K-Means and dimensionality reduction are unsupervised. Evaluation commonly uses accuracy/F1 for classification and MSE for regression; clustering can use inertia, silhouette score and Davies-Bouldin index.

Algorithm / Steps: Compare input labels, tasks, algorithms and evaluation.

Output: Supervised: classification/regression. Unsupervised: clustering/structure discovery.

Result: The two approaches are distinguished.

Q3) Viva: Prepare definitions, working steps, key terms, complexity/accuracy, and one practical example for the programs above.

SLIP 24 — CS-321-VSC-P PRACTICAL

Foundation of Artificial Intelligence and Machine Learning

Q1) Implement A* Search.

Aim: To find a shortest path using heuristic search.

Theory: A* uses f(n)=g(n)+h(n). The slip image provides a graph with heuristic values; the code demonstrates the same method on an equivalent weighted graph.

Algorithm / Steps: Initialize start; select smallest f; expand neighbors; update path cost; stop at destination.

Program:

```
import heapq

graph = {
    "A": [("B", 2), ("C", 4)],
    "B": [("D", 1)],
    "C": [("D", 2)],
    "D": [("E", 3)],
    "E": [("F", 2)],
    "F": []
}
h = {"A": 7, "B": 6, "C": 3, "D": 2, "E": 1, "F": 0}

def a_star(start, goal):
    pq = [(h[start], 0, start, [start])]
    best_g = {start: 0}

    while pq:
        f, g, node, path = heapq.heappop(pq)
        if node == goal:
            return path, g
        for nxt, cost in graph[node]:
            new_g = g + cost
            if new_g < best_g.get(nxt, float("inf")):
                best_g[nxt] = new_g
                heapq.heappush(pq, (new_g + h[nxt], new_g,
                                    nxt, path + [nxt]))
    return None, float("inf")

path, cost = a_star("A", "F")
print("Shortest Path:", " -> ".join(path))
print("Path Cost:", cost)
```

Output: Shortest path and path cost are printed.

Result: A* shortest-path search is implemented.

Q2) Implement K-Means clustering.

Aim: To group observations into K clusters.

Theory: K-Means assigns points to the nearest centroid and repeatedly updates centroids until convergence.

Algorithm / Steps: Load numeric data; optionally standardize; fit KMeans with chosen K; print labels and cluster centers.

Program:

```
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

X,_=make_blobs(n_samples=200,centers=3,cluster_std=1.5,random_state=42)
model=KMeans(n_clusters=3,random_state=42,n_init=10)
labels=model.fit_predict(X)
```

```
print("Cluster labels:",labels)
print("Cluster centers:\n",model.cluster_centers_)
```

Output: Cluster labels and centers are printed.

Result: K-Means groups the supplied data into three clusters.

OR — Alternative Q2

Q2 (OR)) Classify MNIST images using CNN with the specified settings.

Aim: To train a CNN on MNIST with normalization, 80:20 split, two convolution layers, dropout 20%, 2×2 pooling, padding=1, learning rate=.01, 10 epochs and Adam.

Theory: MNIST contains 28×28 grayscale handwritten digits. Normalization scales pixels to 0–1. A CNN learns spatial features using convolution and pooling.

Algorithm / Steps: Load and normalize MNIST; reshape to add channel dimension; split 80:20; build two Conv2D layers with padding='same'; dropout=.20; pool 2×2; compile with Adam(.01); train 10 epochs; evaluate.

Program:

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Conv2D,MaxPooling2D,Dropout,Flatten,Dense
from tensorflow.keras.optimizers import Adam

(x_train,y_train),(x_test,y_test)=tf.keras.datasets.mnist.load_data()
x=tf.concat([x_train,x_test],axis=0).numpy().astype("float32")/255.0
y=tf.concat([y_train,y_test],axis=0).numpy()

x=x[...,None]
n=len(x); split=int(.8*n)
x_train,x_test=x[:split],x[split:]
y_train,y_test=y[:split],y[split:]

model=Sequential([
    Conv2D(32,(3,3),padding="same",activation="relu",input_shape=(28,28,1)),
    MaxPooling2D((2,2)),
    Dropout(.20),
    Conv2D(64,(3,3),padding="same",activation="relu"),
    MaxPooling2D((2,2)),
    Dropout(.20),
    Flatten(),
    Dense(128,activation="relu"),
    Dense(10,activation="softmax")
])
model.compile(optimizer=Adam(learning_rate=.01),
              loss="sparse_categorical_crossentropy",metrics=["accuracy"])
model.fit(x_train,y_train,epochs=10,batch_size=128,validation_split=.1,verbose=1)
loss,acc=model.evaluate(x_test,y_test,verbose=0)
print("Test Accuracy:",acc)
```

Output: Test accuracy is printed after training.

Result: The MNIST CNN is trained and evaluated on the test set.

Q3) Viva: Prepare definitions, working steps, key terms, complexity/accuracy, and one practical example for the programs above.

SLIP 25 — CS-321-VSC-P PRACTICAL

Foundation of Artificial Intelligence and Machine Learning

Q1) Implement AO* Algorithm.

Aim: To demonstrate cost evaluation in an AND-OR graph.

Theory: AO* is an informed search algorithm for AND-OR graphs. At OR nodes one alternative is selected; at AND nodes all required child solutions contribute to the cost.

Algorithm / Steps: Assign heuristic/edge costs; expand promising nodes; compute OR as minimum and AND as sum; update parent cost; continue to a solution graph.

Program:

```
# Simplified AO* evaluation for the AND-OR graph shown in the slip.
# OR node: choose minimum child cost. AND node: add required child costs.

graph = {
    "A": {"type":"OR","children":[("B",2),("C",3)]},
    "B": {"type":"AND","children":[("D",1),("E",4)]},
    "C": {"type":"OR","children":[("F",2),("G",5)]},
    "D": {"type":"OR","children":[("H",2),("I",3)]},
    "E": {"type":"OR","children":[("J",1)]}
}
cost={"F":0,"G":0,"H":0,"I":0,"J":0}

def ao(node):
    if node not in graph:
        return cost.get(node,0)
    data=graph[node]
    vals=[edge+ao(child) for child,edge in data["children"]]
    return sum(vals) if data["type"]=="AND" else min(vals)

print("AO* estimated optimal cost:",ao("A"))
```

Output: AO* estimated optimal cost is printed.

Result: The supplied AND-OR structure is evaluated using AO* cost rules.

Q2) Implement Minimax for a two-player game.

Aim: To select the optimal value from the supplied MAX-MIN game tree.

Theory: MAX chooses the largest child value; MIN chooses the smallest. The root's minimax value is the optimal game value.

Algorithm / Steps: Recursively evaluate leaves; MIN takes minimum; MAX takes maximum; return root value.

Program:

```
def minimax(depth,node,maximizing,values):
    if depth==2:
        return values[node]
    if maximizing:
        return max(minimax(depth+1,2*node,False,values),
                   minimax(depth+1,2*node+1,False,values))
    return min(minimax(depth+1,2*node,True,values),
               minimax(depth+1,2*node+1,True,values))

values=[3,5,2,9]
print("Optimal value:",minimax(0,0,True,values))
```

Output: For leaf values [3,5,2,9], Optimal value: 5.

Result: The Minimax algorithm selects the optimal game-tree value.

OR — Alternative Q2

Q2 (OR)) Implement Forward Chaining inference.

Aim: To derive conclusions from facts and rules.

Theory: Forward chaining starts from known facts and repeatedly fires rules whose premises are satisfied.

Algorithm / Steps: Initialize facts; repeatedly check rule premises; add conclusions; stop when no new fact is produced.

Program:

```
facts = {"fever", "cough"}
rules = [
    ({"fever", "cough"}, "flu"),
    ({"flu"}, "take_rest"),
    ({"flu", "body_pain"}, "consult_doctor")
]

changed = True
while changed:
    changed = False
    for premises, conclusion in rules:
        if premises.issubset(facts) and conclusion not in facts:
            facts.add(conclusion)
            changed = True

print("Derived facts:", sorted(facts))
```

Output: Derived facts include flu and take_rest.

Result: Forward chaining inference is completed.

Q3) Viva: Prepare definitions, working steps, key terms, complexity/accuracy, and one practical example for the programs above.

Note: The practical slips are reproduced in their original Q1/Q2/OR structure. Where a slip leaves random-state or dataset-generation details unspecified, the code uses a fixed random_state only to make the run reproducible. Replace file paths such as country_data.csv or image folders with the files/folders available in your lab.